



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Camila Raquel Marcos Medinilla **Carné:** 1120525

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?

Es un estilo de arquitectura que se utiliza para la comunicación e intercambio de datos entre dos sistemas web. Usa recursos que se identifican con IDs.

2. ¿Qué es un endpoint?

Es una ruta específica de una URL a la que se accede para interactuar con un recurso determinado dentro de la API. Es la dirección exacta con la que el cliente conecta para realizar una acción o solicitar datos.

3. ¿Qué diferencia existe entre una URL y un endpoint?

La URL es la dirección completa y un endpoint es una sección específica de un URL.

4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?

Los métodos HTTP indican al servidor qué acción se quiere realizar sobre un recurso de la API.

5. ¿Qué es JSON y por qué es común en las APIs?

JavaScript Object Notation es un formato de texto utilizado para estructurar y transferir datos. Es el estándar más común en las APIs porque es fácil de leer y escribir tanto para los humanos como para las máquinas. Y es compatible con los lenguajes de programación modernos.

6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.

Es un número enviado por el servidor que indica el resultado de la petición realizada por el cliente.

200 significa OK o éxito.

404 significa que no se pudo encontrar el recurso solicitado (not found)

500 significa que hubo un error del servidor.

7. ¿Qué información se puede encontrar en la respuesta de una API?

Código de estado HTTP: 200, 404, 500

Encabezados: que proveen información sobre la respuesta como tipo de contenido o fecha.

Cuerpo: los datos reales solicitados, usualmente en formato JSON.

8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?

Permite diseñar y validar la lógica de la API sin necesidad de escribir código de cliente. Permite verificar rápidamente si las rutas responden como se espera.

9. ¿Qué es y para qué sirve Swagger UI?

Es una herramienta visual que genera documentación interactiva para una API a partir de su especificación. Sirve para que los desarrolladores entiendan cómo funciona la API, qué parámetros requiere cada ruta, qué tipo de respuestas devuelve, e incluso permite probar los endpoints directamente desde el navegador.



10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?
Postman actúa como un entorno completo de desarrollo para APIs.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.



- Indique el método HTTP utilizado.
GET
- Indique qué parámetro necesita la solicitud.
El petId
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
Código 200 – OK.
- Explique con sus palabras qué información contiene la respuesta.
Información del producto con el ID 3, como el nombre, la categoría y el estatus.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/3' \
  -H 'accept: application/json'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet/3
```

Server response

Code Details

200

Response body

```
{
  "id": 3,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "doggie",
  "photoUrls": [
    "azuoktblivusfqgqjcmkufjylwfnac",
    "azuoktblivusfqgqjcmkufjylwfnac",
    "azuoktblivusfqgqjcmkufjylwfnac"
  ],
  "tags": [
    {
      "id": 7453702480964536000,
      "name": "yqfctxrgaksreecnxdtpvyslbvtdcvj"
    },
    {
      "id": 7453702480964536000,
      "name": "yqfctxrgaksreecnxdtpvyslbvtdcvj"
    }
  ],
  "status": "sold"
}
```

Download



3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

POST /pet Add a new pet to the store.

Add a new pet to the store.

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Create a new pet in the store

Edit Value | Schema

```
{
  "id": 1120525,
  "name": "chucho",
  "category": {
    "id": 1120525,
    "name": "perro"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "disponible"
}
```

Execute

Responses

Curl

```
curl -X 'POST' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 1120525,
    "name": "chucho",
    "category": {
      "id": 1120525,
      "name": "perro"
    },
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 0,
        "name": "string"
      }
    ],
    "status": "disponible"
  }'
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

Code Details

200

Response body

```
{
  "id": 1120525,
  "category": {
    "id": 1120525,
    "name": "perro"
  },
  "name": "chucho",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "disponible"
}
```

Download



Name	Description
petId * required integer(\$int64) (path)	ID of pet to return 1120525

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/1120525' \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/1120525

Server response

Code

200

Response body

```
{
  "id": 1120525,
  "category": {
    "id": 1120525,
    "name": "perro"
  },
  "name": "chuchito",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "disponible"
}
```



Download

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.
- ¿Qué método HTTP utilizó y por qué?
POST/pet/petId. Porque me permite buscar el elementos que quiero actualizar por ID y cambiar parámetros de forma más sencilla.
- ¿Qué datos fueron enviados en el body?
El ID de la mascota que quería modificar.
- ¿Qué código HTTP devolvió la API?
200
- ¿Cómo comprobó que el cambio realmente ocurrió? Volviendo a usar GET y viendo el resultado.

POST

/pet/{petId} Updates a pet in the store with form data.

🔒 ⬆

Updates a pet resource based on the form data.

Parameters

Cancel

Name	Description
petId * required integer(\$int64) (path)	ID of pet that needs to be updated 1120525
name string (query)	Name of pet that needs to be updated tommy
status string (query)	Status of pet that needs to be updated comprado

Execute

Clear



Curl

```
curl -X 'POST' \
  'https://petstore3.swagger.io/api/v3/pet/1120525?name=tommy&status=comprado' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

https://petstore3.swagger.io/api/v3/pet/1120525?name=tommy&status=comprado

Server response

Code

Details

200

Response body

```
{
  "id": 1120525,
  "category": {
    "id": 1120525,
    "name": "perro"
  },
  "name": "tommy",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "comprado"
}
```

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 147
content-type: application/json
date: Fri,02 Oct 2026 12:27:14 GMT
server: Jetty(9.4.53.v20231009)
```

GET

/pet/{petId} Find pet by ID.

Returns a single pet.

Parameters

Cancel

Name	Description
------	-------------

petId * required	ID of pet to return
integer(int64)	
(path)	

1120525

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/1120525' \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/1120525

Server response

Code

Details

200

Response body

```
{
  "id": 1120525,
  "category": {
    "id": 1120525,
    "name": "perro"
  },
  "name": "tommy",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "comprado"
}
```

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 147
content-type: application/json
date: Fri,02 Oct 2026 12:29:57 GMT
server: Jetty(9.4.53.v20231009)
```



5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
El parámetro utilizado es el estatus.
- Explique dónde se envía ese parámetro.
En el URL que se envía.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.
Si se cambia a pending solo muestra un resultado.

GET /pet/findByStatus Finds Pets by status.

Multiple status values can be provided with comma separated strings.

Parameters

Name	Description
status * required	Status values that need to be considered for filter
string	
(query)	available

Execute

Clear

Cancel

Responses

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available' \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/findByStatus?status=available

Server response

Code

Details

200

Response body

```
{
  "id": 12345,
  "category": {
    "id": 1,
    "name": "dogs"
  },
  "name": "fluffy",
  "photoUrls": [],
  "tags": [
    {
      "id": 0,
      "name": "tag1"
    },
    {
      "id": 1,
      "name": "tag2"
    }
  ],
  "status": "available"
},
{
  "id": 123456,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "Dogs"
}
```

Download

Response headers



status * required Status values that need to be considered for filter
string
(query)
pending

Execute Clear

Responses

Curl

```
curl -X 'GET' \
'https://petstore3.swagger.io/api/v3/pet/findByStatus?status=pending' \
-H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/findByStatus?status=pending

Server response

Code Details

200

Response body

```
{
  "id": 100,
  "category": {
    "id": 10000,
    "name": "Perros"
  },
  "name": "Kobe",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 100,
      "name": "Doki"
    }
  ],
  "status": "pending"
}
```

Download

6. Reproducir las operaciones en Postman

Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.

- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID

GET https://petstore3.swagger.io/api/v3/pet/1120525 Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Body Cookies Headers 9 Test Results 200 OK • 217 ms • 509 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "id": 1120525,
3   "category": {
4     "id": 1120525,
5     "name": "perro"
6   },
7   "name": "tommy",
8   "photoUrls": [
9     "string"
10  ],
11  "tags": [
12    {
13      "id": 0,
14      "name": "string"
15    }
16  ],
17  "status": "comprado"
18 }
```



POST <https://petstore3.swagger.io/api/v3/pet> Send

Docs Params Authorization Headers 8 **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON Schema Beautify

```
1 {
2   "id": 13,
3   "name": "perrito",
4   "category": {
5     "id": 1,
6     "name": "cani"
7   },
8   "photoUrls": [
9     "string"
10  ],
11 }
```

Body Cookies Headers 9 Test Results 200 OK · 356 ms · 501 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "id": 13,
3   "category": {
4     "id": 1,
5     "name": "cani"
6   },
7   "name": "perrito",
8   "photoUrls": [
9     "string"
10  ],
11   "tags": [
12     {
13       "name": "tag"
14     }
15  ]
16 }
```

- PUT – Modificar mascota

PUT <https://petstore3.swagger.io/api/v3/pet> Send

Docs Params Authorization Headers 8 **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☐ JSON Schema Beautify

```
1 {
2   "id": 13,
3   "name": "chuchooooo",
4   "category": {
5     "id": 1,
6     "name": "mamam"
7   },
8   "photoUrls": [
9     "string"
10  ],
11 }
```

Body Cookies Headers 9 Test Results 200 OK · 62 ms · 504 B Save Response

{ } JSON Preview Visualize

```
1 {
2   "id": 13,
3   "category": {
4     "id": 1,
5     "name": "mamam"
6   },
7   "name": "chuchooooo",
8   "photoUrls": [
9     "string"
10  ],
11   "tags": [
12     {
13       "name": "tag"
14     }
15  ]
16 }
```

Open Folder Terminal Console Add post-response tests Globals Vault Tools



```
GET https://petstore3.swagger.io/api/v3/pet/findByStatus?status=pending

200 OK - 143 ms - 504 B

[
  {
    "id": 100,
    "category": {
      "id": 10000,
      "name": "Perros"
    },
    "name": "Kobe",
    "photoUrls": [
      "String"
    ],
    "tags": [
      {
        "id": 100,
        "name": "Doki"
      }
    ],
    "status": "pending"
  }
]
```

- DELETE – Eliminar mascota

```
DELETE https://petstore3.swagger.io/api/v3/pet/13

200 OK - 143 ms - 504 B

{
  "message": "Pet deleted"
}
```

- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.



7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.

GET

/pet/{petId} Find pet by ID.

🔒 ⬆

Returns a single pet.

Parameters

Cancel

Name	Description
petId * required integer(int64) (path)	ID of pet to return

Execute

Clear

Responses

Curl

curl -X 'GET' \n 'https://petstore3.swagger.io/api/v3/pet/1120526' \n -H 'accept: application/json'

Request URL

https://petstore3.swagger.io/api/v3/pet/1120526

Server response

Code	Details
404	Error: response status is 404

Response body

can't parse JSON. Raw result:
Pet not found

Download

Response headers

access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 13
content-type: application/json
date: Fri,02 Oct 2026 12:38:56 GMT
server: Jetty(9.4.53.v20231009)

2. Enviar un JSON incorrecto o incompleto.

POST

/pet Add a new pet to the store.

🔒 ⬆

Add a new pet to the store.

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

Create a new pet in the store

Edit Value | Schema

```
{
  "id": 12345,
  "name": "Chicho",
  "category": {
    "id": 120525,
    "name": "perro"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "disponible"
}
```

Execute

Clear



Responses

Curl

```
curl -X 'POST' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": "camila",
    "name": "chucho",
    "category": {
      "id": "112052S",
      "name": "perro"
    },
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": "0",
        "name": "string"
      }
    ],
    "status": "disponible"
  }'
```

Request URL

https://petstore3.swagger.io/api/v3/pet

Server response

Code Details

400 Error: response status is 400

Response body

```
{
  "code": 400,
  "message": "Input error: unable to convert input to io.swagger.petstore.model.Pet"
}
```



Download

Response headers

```
access-control-allow-headers: Content-Type,api_key,Authorization
access-control-allow-methods: GET,POST,DELETE,PUT
access-control-allow-origin: *
access-control-expose-headers: Content-Disposition
content-length: 94
content-type: application/json
date: Fri,02 Oct 2026 12:40:21 GMT
server: Jetty(9.4.53.v20231009)
```

3. Utilizar un parámetro con un valor no esperado.

GET

/pet/{petId} Find pet by ID.

Returns a single pet.

Parameters

Cancel

Name	Description
petId * required	ID of pet to return
integer(\$int64) (path)	hola

Please correct the following validation errors and try again.

- For 'petId': Value must be an integer.

Execute

Clear

Para cada prueba registre:

- Solicitud realizada.
- Código HTTP recibido.
- Respuesta de la API.



8. API Detective

Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.

Usaría POST/pet para registrar una mascota, GET/pet/petId para consultar su información, PUT/pet/petId para actualizar su estado y DELETE/pet/petId para eliminarla.

- Defina el método HTTP correspondiente.

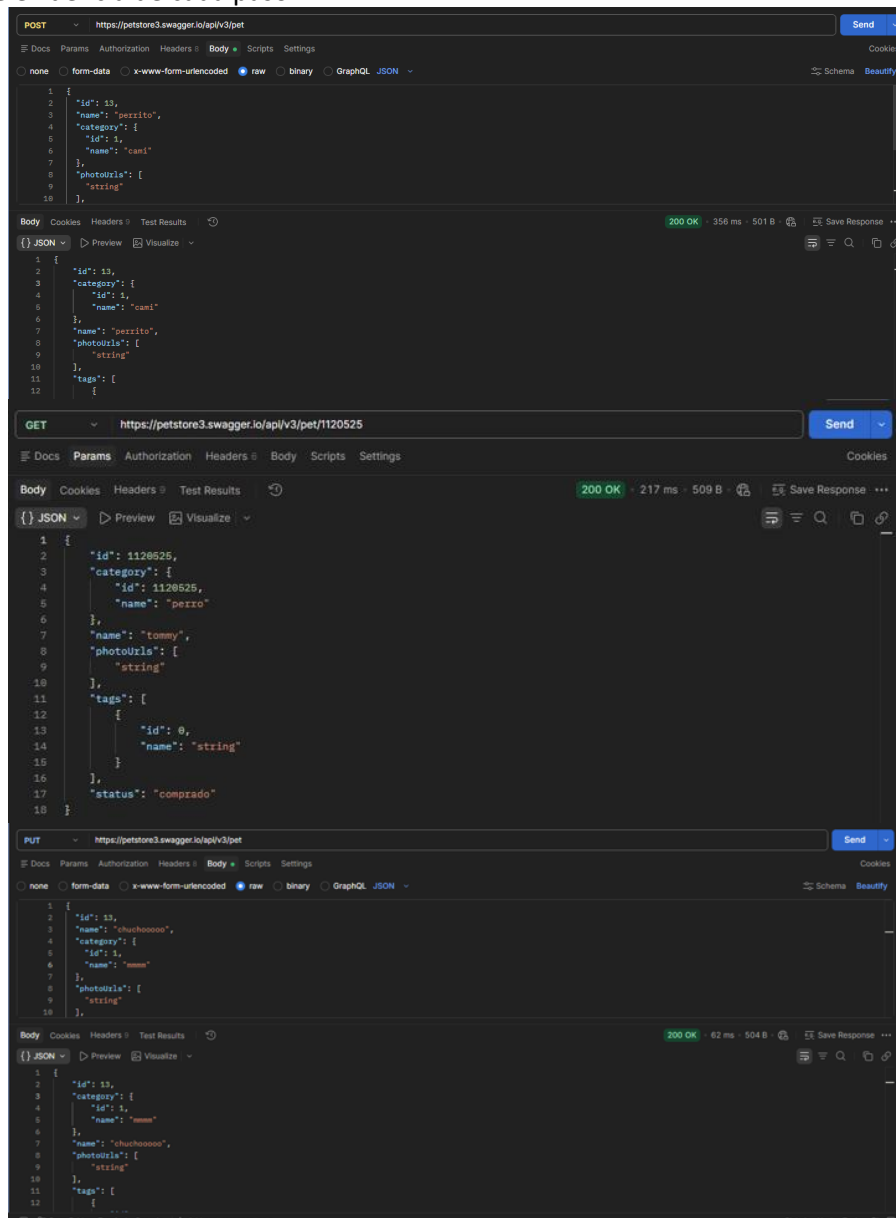
POST para registrar una mascota, GET para consultar su información, PUT para actualizar su estado y DELETE para eliminarla.

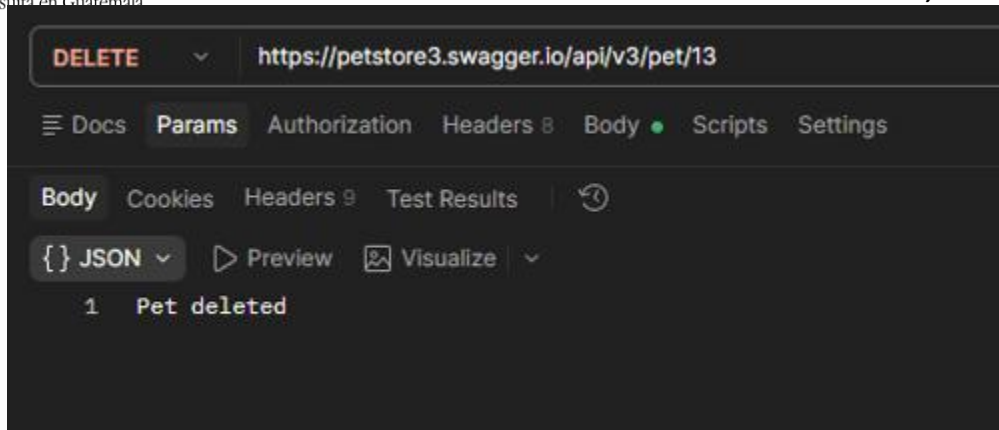
- Indique qué respuesta debe analizar el programa.

Si los recursos con ese ID existen y todo se ingresa correctamente debería devolver 200 de OK. En caso contrario, podría regresar 404 de Not Found o 400 de Bad Request.

- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.





Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.